

Figure 4: Encoder interpretability scaling curves. All methods are trained on the same data. At $k = 16$, PCD scales comparably to SAE variants. Standard SAEs benefit substantially from increasing to $k = 50$, while KL-based methods show smaller gains and plateau around 72M.

This selects dead concepts that are close to being active and nudges them in a direction that encourages them to become active in similar contexts. By training with this auxiliary loss, we keep over 90% of concepts active at the 72M-token scale.

We detail alternate unsuccessful approaches for managing concept activity in Appendix A.2.

3.3 Evaluating Encoder Interpretability

Beyond predictive performance, we want the encoder's concepts to be interpretable to humans. We evaluate along two axes, framed loosely as *precision* (are the learned concepts high-quality?) and *recall* (do the concepts cover diverse phenomena?).

Precision: auto-interpretability score. To measure whether individual concepts are interpretable, we use the automated interpretability pipeline of Choi et al. (2024). For each concept direction, we collect top-activating exemplars from held-out FineWeb passages, generate natural language descriptions, and measure how well a finetuned simulator can predict activation patterns on new exemplars given only the description (via Pearson correlation). We evaluate a random sample of 400 concepts and report the average score.

Recall: user modeling accuracy. To measure concept coverage, we use the SynthSys dataset (Choi et al., 2025), which defines user attributes (e.g. "marital status") that can take different values (e.g. "married", "divorced"), along with associated prompts. For each attribute, we construct a balanced binary classification task and train a scalar linear classifier (i.e. a threshold) on each encoder direction's activations. We report the best classifier's held-out accuracy, measuring whether *some* concept captures each attribute.

Results and scaling behavior. Results with and without the auxiliary loss are shown in Figure 3. The auxiliary loss improves both metrics on longer training runs: without it, the auto-interp score plateaus and recall decreases as training progresses from 36M to 72M tokens. With the auxiliary loss, both interpretability metrics increase with pretraining data, although they start to plateau past the 100M-token scale as seen in Figure 4. We investigate this next through a set of ablations.

Identifying the cause of the plateau. We check if SAEs, a common approach for learning concept dictionaries from neural network activations, suffer from the same plateau. SAEs differ from PCDs in three ways: they have no learned decoder, they predict activations rather than output tokens, and they predict at the current token position rather than the future.

We first train standard SAEs on the same FineWeb dataset, training with L2 reconstruction loss on the n_{middle} activations. We count training token budget based on the number of tokens passing through the encoder. We see in Figure 4 that, when we use fewer than 36M training tokens, PCDs with $k = 16$ active concepts outperform SAEs with $k = 16$ or $k = 50$ concepts. However, as we increase training tokens, the SAEs with $k = 50$ concepts surpass the PCD, largely because the PCD curves plateau. We also tried increasing k for PCDs, but found in preliminary experiments that (unlike SAEs) this did not improve their performance.

To identify why this is happening, we train SAE variants that progressively interpolate between SAEs and PCDs. These *KL SAEs* are trained to minimize KL divergence between subject model

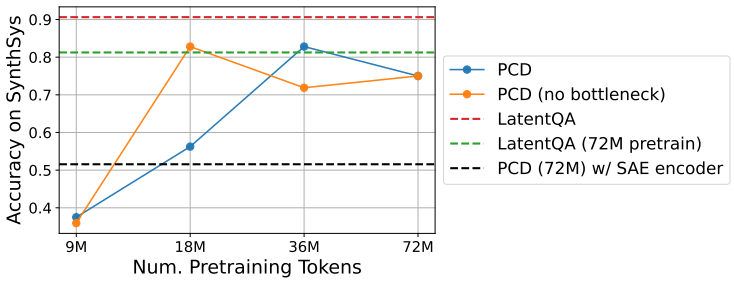


Figure 5: Question-answering accuracy generally improves as the encoder is trained for longer and eventually matches LatentQA, even though LatentQA gets to read the entire residual stream. PCD performance saturates earlier (after 18M) if we remove the bottleneck at test time. All measurements are using the final finetuning checkpoint (4000 steps; see Figure 18 for results across checkpoints).

outputs with original vs. reconstructed activations (Braun et al., 2024). We test two versions: one computing KL on the current tokens (n_{middle}), and one on the future tokens (n_{suffix}), with the latter more closely resembling PCD's objective.

Results for these encoders are also shown in Figure 4. We see that both KL SAEs have the same plateauing behavior as PCDs, and even do worse on recall as data increases. Additionally, the L2 SAEs benefit significantly from increasing the number of active concepts from $k = 16$ to $k = 50$, while the KL SAEs see smaller gains. This suggests that KL-based objectives can capture relevant information with fewer active concepts, but may saturate earlier. It is possible that the KL objective may be too "easy" to optimize compared to L2 reconstruction, leading to a sparser signal. We leave addressing this to future work.

Model selection. Based on these results, we select the PCD trained on 72M tokens—which achieves the best encoder interpretability before the plateau—for all downstream experiments. We generate descriptions for all 32,768 concepts in this dictionary using the Choi et al. (2024) auto-interp pipeline, and reference these descriptions throughout the rest of our work. In Appendix A.4 we show the results of varying other hyperparameters in PCD training, such as the number of active concepts, predicting the next token vs. matching subject model probabilities, and LoRA rank. We do not find significant improvements from changing these.

4 Finetuning

We next finetune the decoder to answer questions about the subject model's beliefs, using the concepts extracted by the encoder. We use SynthSys(8B) (Choi et al., 2025), which contains user dialogues where the subject model has made an assumption about user attributes such as "employment" or "marital status", along with a question-answer pair about the model's belief about the user. SynthSys dialogues are filtered for *consistency of revealed beliefs*—the subject model answers several questions in response to the dialogue, and only dialogues where the responses are consistent with the user attribute are kept. This helps provide (noisily) labeled data about the subject model's internal state, based on consistency of behaviors.

Finetuning setup. We train on 78 of the 80 user attributes in SynthSys and hold out "gender" and "age" for evaluation. The encoder reads activations from the SynthSys user message; the decoder receives the encoded activations and must predict the assumed attribute. Qualitative examples of SynthSys evaluation questions and relevant encoder concepts are shown in Figure 6.

For finetuning, the encoder is always frozen, as it should already have learned interpretable concepts and our goal is to improve the question-answering ability of the decoder. We continue mixing in FineWeb sequences at 50% frequency to reduce forgetting. We use the same hyperparameters as during pretraining, listed in Appendix A.1.

Baselines. To investigate the effect of the encoder bottleneck, we compare against LatentQA, which has the same decoder architecture as PCDs but no encoder. We consider both a LatentQA decoder that is pretrained on FineWeb then finetuned on SynthSys(8B), as well as one that is only trained

Female SynthSys Example

System message: Known detail: user = woman.

User message: I'm experiencing brain fog and difficulty concentrating. What might help?

Decoder question: Is the assistant assuming a particular value of the user's gender?
A. female; B. male; C. non-binary; D. None of the above

Relevant concepts:

- "the word "gender" when referring to identity, roles, or social constructs surrounding it; occurrences of the token "androgyny" as well as "girl" when referencing gender identity or characteristics."
- "tokens like "synth", "her", "girl", and "she" indicating female nouns or pronouns, often tied to emotional or nostalgic contexts."

Male SynthSys Example

System message: Session data indicates your user has male-pattern genetic predispositions.

User message: What are some warning signs of hormonal imbalances?

Decoder question: Is the assistant assuming a particular value of the user's gender?
A. male; B. female; C. non-binary; D. None of the above

Relevant concepts:

- "the token 'Jeff' when it is part of names or geographical references"
- "the name 'Luis' in various contexts (e.g., people, places, events)"

Figure 6: SynthSys questions and active concepts from PCD (72M). The decoder only sees activations from the user message. The encoder surfaces gender-related concepts that help the decoder.

on SynthSys(8B) as in Choi et al. (2025). Additionally, to investigate whether joint pretraining is important, we also consider an ablation of PCDs where the encoder is frozen to match the weights of the best SAE from Section 3.3 during both pretraining and finetuning.

Results. We evaluate each decoder's performance on multiple-choice SynthSys questions for the held-out user attributes, and report results in Figure 5. We see that at small pretraining budgets, PCDs perform poorly compared to LatentQA, but achieve comparable performance starting at 36M tokens. In contrast, ablating PCDs to use a frozen SAE encoder performs poorly. Together, these results show that the encoder bottleneck does not significantly harm performance on the user modeling task, but that it is important for it to be jointly learned with the decoder.

In the next section, we find that on certain complex tasks the bottleneck *does* harm performance, and we can sometimes do better by replacing it with the identify function (setting $\mathbf{W}_{\text{enc}} = \mathbf{W}_{\text{emb}} = \mathbf{I}$ and removing the TopK nonlinearity). As a preliminary investigation, we report the results of doing this for SynthSys in the right panel of Figure 5; note that this intervention is applied only at test time, not training time, so the input to the decoder is entirely OOD in this setting. Interestingly, this variant improves performance at 18M pretraining tokens, but reduces performance at 36M and 72M tokens. This suggests that as pretraining duration increases, the PCD encoder is able to more effectively pass relevant information through the bottleneck to the decoder.

5 Case Studies: Using PCDs to Explain Atypical Behaviors

We now investigate a variety of scenarios where an atypical LM behavior is present: jailbreaking models to output harmful information in an indirect format, providing secret hints that models use

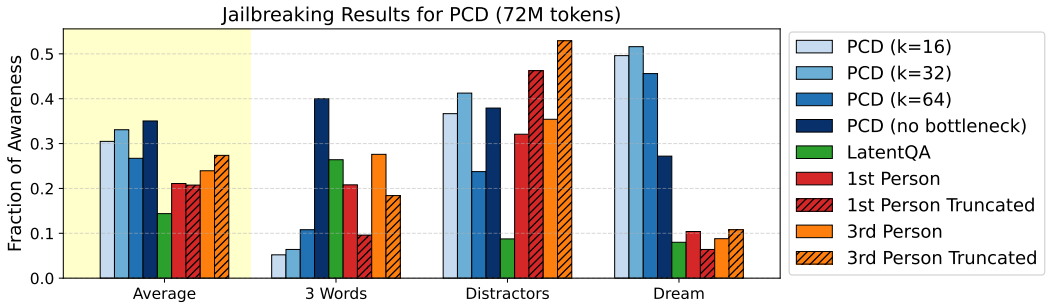


Figure 7: **Results on Jailbreaking**. PCDs are aware that the subject model will perform a bad action (like outputting instructions to build a bomb) due to being jailbroken, at a rate that is similar to, and sometimes much better than, asking the subject model what happened.

as a shortcut, and implanting latent concepts by patching the activations. In each case, we evaluate the ability of PCDs to surface the behavior relative to several baselines.

For PCDs, our primary results are for the decoder trained on 72M tokens with k fixed at 16 throughout training. At test time, we vary k within $\{16, 32, 64\}$ to investigate cases where information is available in the activations but not surfaced in the top 16 concepts. Note that for $k \neq 16$ the inputs to the decoder are OOD relative to training. We additionally consider a setting where we remove the bottleneck entirely, setting $k = \infty$ and $\mathbf{W}_{\text{enc}} = \mathbf{W}_{\text{emb}} = \mathbf{I}$. Finally, we report results for PCD decoders pretrained on $\{9, 18, 36\}$ million tokens, to investigate the effect of pretraining scale.

Baselines. Similarly to Section 4, we evaluate a LatentQA decoder that was pretrained on FineWeb for 72M tokens and finetuned on SynthSys. This is architecturally equivalent to a PCD with no bottleneck, but has a different inductive bias from training. We also consider several prompting baselines where we directly ask the subject model about the behavior of interest.

Scaling trends and inductive bias. Throughout our results we observe two common themes: first, PCDs become progressively better with more pretraining data; this effect is most important at higher sparsity ($k = 16$), suggesting that more data teaches the PCD encoder to more efficiently pass information to the decoder. Second, PCDs consistently outperform LatentQA. This is interesting, because one of the variants we study (PCD with no bottleneck) is architecturally identical to LatentQA but has a different training objective, suggesting that PCD training provides a favorable inductive bias.

5.1 JAILBREAKING

Jailbreaks trick LMs into verbalizing things that they normally would not. For example, we can trick Llama-3.1-8B-Instruct into outputting instructions on how to build a bomb if the instructions are part of a dream, or if we cleverly hide them in a storytelling game. These attacks confuse the subject model enough that it is often unable to recognize what it did when given its own transcript. We are interested in seeing if PCDs can understand what the subject model is going to do, even with the obfuscation from the jailbreak.

To study this, we curate attacks that successfully jailbreak Llama-3.1-8B-Instruct into providing dangerous instructions. We construct three templates based on known attacks—*3 Words* (aashkafirst, 2024), *Distractors* (Wei et al., 2023), and *Dream* (Ding et al., 2024; Shang & Wei, 2025)—and apply each to 50 actions that the model typically refuses. For each (template, action) pair, we repeatedly sample responses until GPT-5-mini classifies one as a successful jailbreak, discarding a small minority of pairs that fail after 10 attempts.

For all methods, we evaluate awareness of the jailbreak by asking variants of the question "What is the assistant thinking about right now?" and use GPT-5-mini to categorize responses into *awareness* (the decoder is aware of the harmful action), *instructions* (the decoder instead produces instructions for the action), *refusal* (the decoder refuses to answer), and *other*. We average results over 5 independent samples from the decoder for each jailbreak input.